

Sustainable society with a touchless solution using UbiMouse under the pandemic of COVID-19

Daisuke Akagawa*
akagawa@chinou.co.jp
Chinougiyutsu
Osaka, Osaka-fu, Japan

Junichi Takatsu*
takatsu@chinou.co.jp
Chinougiyutsu
Osaka, Osaka-fu, Japan

Ryoji Otsu*
otsu@chinou.co.jp
Chinougiyutsu
Osaka, Osaka-fu, Japan

Seiichi Hayashi*
s.hayashi@chinou.co.jp
Chinougiyutsu
Osaka, Osaka-fu, Japan

Benjamin Vallet*
vallet@chinou.co.jp
Chinougiyutsu
Osaka, Osaka-fu, Japan

ABSTRACT

This paper introduces a new artificial intelligence software which is capable of controlling devices using fingers in the air. With Ubi-mouse, touch-panels, restaurant ordering systems, ATM systems, and etc., which are commonly used by various people in public, can be contact-less devices. These touch-less devices, especially under the harsh conditions of COVID-19, are desired to prevent infections mediated by touch devices. Also, these devices cannot be used while wearing gloves due to their touch sensing fault. Thus, in fields using gloves, there is a demand for non-contact device operation. To satisfy these demands, we have developed “UbiMouse”. This is an AI software that allows you to operate the device by moving your fingers toward the device. In the AIs in UbiMouse, a convolution model and a regression model are used to identify fingers’ features from camera footage and to estimates the position of a detected finger, respectively. We demonstrate an operation of UbiMouse without contact. Along with this operation in the air, a mouse cursor is guided to the specified location with high accuracy.

CCS CONCEPTS

• **Hardware** → **Emerging tools and methodologies**; • **Computing methodologies** → **Motion capture**.

KEYWORDS

non-contact, deep learning, AI based sensor, convolutional networks, regression, cameras, COVID-19, countermeasures

ACM Reference Format:

Daisuke Akagawa, Junichi Takatsu, Ryoji Otsu, Seiichi Hayashi, and Benjamin Vallet. 2021. Sustainable society with a touchless solution using UbiMouse under the pandemic of COVID-19. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Emerging Technologies*

*Authors contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGGRAPH '21 Emerging Technologies, August 9–13, 2021, Virtual Event, USA

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8364-6/21/08...\$15.00

<https://doi.org/10.1145/3450550.3470532>

(*SIGGRAPH '21 Emerging Technologies*), August 9–13, 2021, Virtual Event, USA. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3450550.3470532>

1 INTRODUCTION

Nowadays, devices equipped with touch panels have been used in various areas such as ordering terminals in restaurants, ATMs in banks, and digital signage etc.. In these devices, objects displayed in their screens can be selected intuitively by simply touching the part of the device that needs to be operated, and, since they do not require complicated operations, anyone can use them without knowing how to use them. In addition, these devices’ introduction allows industries to cut necessity to assign personnel to their tasks, which has a significant impact on reducing labor costs. According to the survey [Corporation 2014], the number of self-checkouts is estimated to be 1.2 million worldwide in 2025, and it is expected that contact-type terminals will be used more rapidly in the future.

However, due to the world-wide spread of COVID-19, the situation for touch displays have been changed. People avoid the use of terminals that require contact operation. Especially in public, these terminals are used by an unspecified number of people continuously. This makes it difficult to keep touch screens clean. One possible solution for any company is to sanitize their surface frequently. However, this is very costly.

There is another demand for contact-less control for a specific situation. When gloves are worn on hands, such like workers in some workplaces for safety, an usual touch function may not respond. To use this, removing gloves is cumbersome, and, if those gloves are not allowed to be removed, it is not possible anymore to operate touch panels. Although some touch panels allow to use the touch function with gloves on, it is still difficult to continue to use it due to the possibility of a display getting dirty.

To address these problems, we have developed UbiMouse, which can change a contact device into a contactless device. UbiMouse has two versions; sensor type and software type. The sensor type can be used as a contactless terminal by connecting to an existing device via USB. The software type installs software to a device, and that software operates the terminal.

This operation requires a webcam, such like one already installed to laptops. Currently, UbiMouse is being introduced in restaurants and digital signage as shown in the Figure 1. For both types, they enable novel contactless operations while maintaining the current



Figure 1: Device type UbiMouse



Figure 2: Software type UbiMouse 1



Figure 3: Software type UbiMouse

system in use. For this reason, it is possible to deactivate one of functions, contact and non-contacts, to avoid functional conflicts. Also, to avoid user inconvenience, UbiMouse is designed to be capable of using various screen sizes.

2 RELATED WORK

Google provides MediaPipe which recognizes hand pose. It detects the joints of hand [Zhang et al. 2020]. Leap Motion [Limited [n.d.]] recognizes the hand gesture. Additionally in this study [Mujahid et al. 2021], hand gesture can be recognized using deep learning.

3 DATASET

3.1 SUMMARY

In developing UbiMouse, we created a specific dataset. This dataset uses the image as input and the coordinates of the display that the user points at as label.

The input image was taken using a commonly used webcam. The image size at the time of shooting was (480, 640). Since the image size is changed by preprocessing before input, the image taken by any camera can be used for teaching the model.

Labels were created with our own dedicated software. An image with dots only in a specific place is displayed on the screen, and a finger is pointed at that position to take a picture of the user at that time. You can create a dataset by pairing the captured image with the coordinates where the dots were struck. In this time, we create 1,416 data, and 20% of them were used for validation.

3.2 PREPROCESSING

The following preprocessing was applied to each image and label for the data in 3.1.

The following processing is applied:

- Randomly adjust the brightness value and contrast
- Resize to an image with height of 480 pixel and width of 640 pixel

Assuming that the width of the image is W and the height is H , the range that the pixel coordinates (horizontal direction w , vertical direction h) can take is $w = [0, W - 1]$, $h = [0, H - 1]$. Since the predicted coordinates output by the regression model this time take a numerical value between 0 and 1, the pixel coordinates are converted to (w', h') based on the following command. The process of converting the range of numbers is applied to the label.

$$h' = \frac{h}{H - 1} \quad (1)$$

$$w' = \frac{w}{W - 1} \quad (2)$$

By applying (1) and (2) formulas, both h and w are converted to h' and w' , which are in the range $[0, 1]$ respectively. Also, since the regression output of the model is in the range $[0, 1]$, use the following convention to convert to the corresponding pixel coordinates.

$$h_{pred} = h'_{pred}(H - 1) \quad (3)$$

$$w_{pred} = w'_{pred}(W - 1) \quad (4)$$

h'_{pred}	predicted vertical pixel from regression model
h_{pred}	corrected vertical pixel coordinate
w'_{pred}	predicted horizontal pixel coordinate from regression model
w_{pred}	corrected horizontal pixel coordinate

Calculating the pixel coordinates based on (3) and (4) formulas gives a floating point number. However, since the pixel coordinates must be specified as an integer value, the calculation result is rounded to an integer type.

4 METHOD

4.1 STRUCTURE

UbiMouse consists of two part shown in **Figure 4**. First part is a convolutional model for extracting fingers' features and second part is regression model for predicting position which user points at.

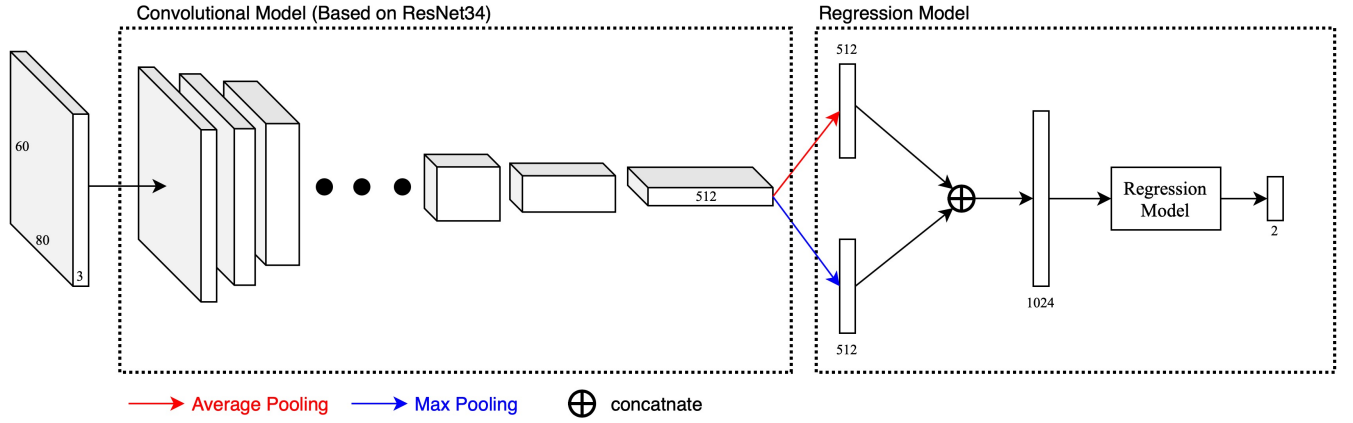


Figure 4: UbiMouse model structure

4.2 CONVOLUTIONAL MODEL

Convolutional model shown in **Figure 4** is based on ResNet34 [He et al. 2015] which is pretrained based on ImageNet [Russakovsky et al. 2015]. The final layer of ResNet34 is a fully-connected layer which has 1,000 nodes. In this case, we removed the layer. By passing input the image, we obtain the output tensor which has 512 channels.

4.3 REGRESSION MODEL

Regression model shown in **Figure 5** is connected after the UbiMouse convolution model. First, we need to convert the output of the convolution model into a format that can be input to the regression model. The output of the convolution model is a tensor with 512 channels. Apply Average Pooling and Max Pooling to the tensor to convert it to a one-dimensional tensor with 512 nodes. The results of both pooling layers are then combined into a 1024-dimensional one-dimensional tensor. Enter the tensor in Figure 3. In this regression model, batch normalization is applied to the hiding place, and dropout is applied to the middle layer. Dropout applies to 102 nodes, which is 20% of the 512. Nodes in the middle tier.

As a result of learning with the regression model, the estimated values of the horizontal and vertical coordinates are output. During operation, this output is converted based on equation (2), converted to display coordinates, and used.

4.4 EVALUATION METRIC

We have used mean squared error loss (MSE) evaluate model's performance. MSE will be calculated according to following equation.

$$MSE = \frac{1}{n} \sum_i \left\{ \left(h'_i - h'_{i,pred} \right)^2 + \left(w'_i - w'_{i,pred} \right)^2 \right\} \quad (5)$$

n	batch size
$h'_{i,pred}$	predicted vertical pixel coordinate
h'_i	true vertical pixel coordinate
$w'_{i,pred}$	predicted horizontal pixel coordinate

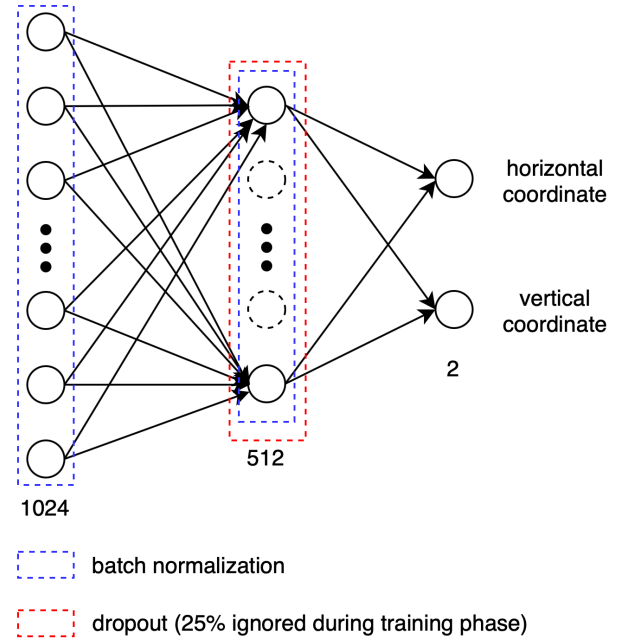


Figure 5: Regression part structure

w'_i true horizontal pixel coordinate

4.5 LEARNING RATE ADJUSTMENT

An LR range test [Smith 2017] was performed to set an appropriate learning rate. By using the LR range test, we can estimate the appropriate learning rate for learning. In addition, the learning rate was variably adjusted by using the Cycle Learning Rate during learning. The result of this LR range test is shown in **Figure 6**. Generally, it is better to set a value slightly smaller than the position where Loss is the lowest value, and this time we set it to 2×10^{-4} .

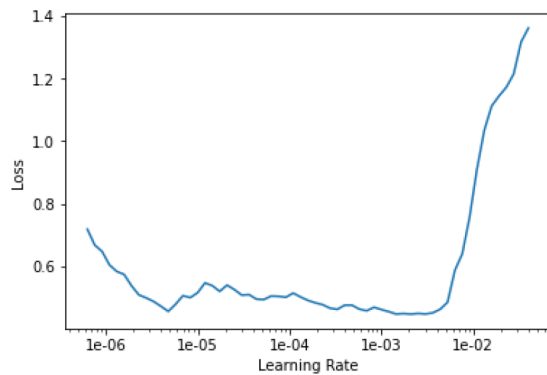


Figure 6: Loss transition

4.6 TRAINING PARAMETERS

In learning UbiMouse, the learning was done in 60 epochs and the batch size was set to 16. And Adam was used as an optimizer.

5 RESULT

As a result of training using the data set created in 3.1, the training loss was transition shown in **Figure 7**. We confirmed that the loss decreased as the learning progressed, and that the learning progressed. In addition, as a result of using it by users who are not included in the learning data, we received the result that the mouse pointer moved in the direction in which the finger was moved and moved to the specified location.

6 CONCLUSION AND FUTURE WORK

In this paper, we reported on the data set creation method, model structure, learning method and results for the software type UbiMouse. As a result of using the created model by many users, we received positive reports that the mouse pointer can be moved to the desired location and that the operability is the same as the contact operation even if it is not in contact. Currently, UbiMouse has been introduced by a large number of users as shown in the introduction, and in the future, we plan to customize the model, such as weight reduction, so that it can be used by a wider variety of users with non-powerful devices.

It is difficult to make a quantitative evaluation when UbiMouse is actually used by untraceable users. However we believe it is necessary to develop more user-friendly software by creating a large number of test cases in the future.

ACKNOWLEDGMENTS

This work was technically supported by Kees Geerligs.

REFERENCES

- NCR Corporation. 2014. *SELF-CHECKOUT: A GLOBAL CONSUMERT PERSPECTIVE*. NCR Corporation.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Deep Residual Learning for Image Recognition. arXiv:1512.03385 [cs.CV]
- Ultra Limited. [n.d.]. *Leap Motion Controller*. Retrieved June 14, 2021 from <https://www.ultraLeap.com>
- Abdullah Mujahid, Mazhar Javed Awan, Awais Yasin, Mazin Abed Mohammed, Robertas Damaševičius, Rytis Maskeliūnas, and Karrar Hameed Abdulkareem. 2021.

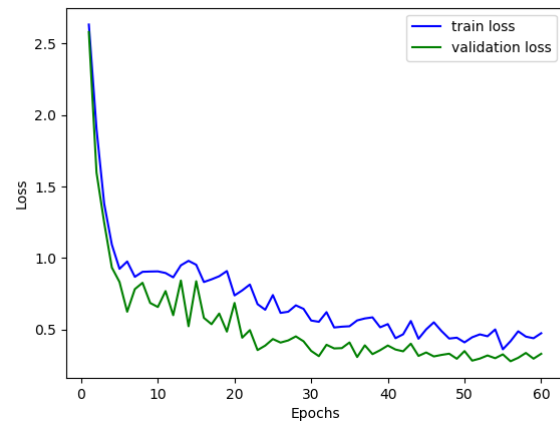


Figure 7: Training result

- Real-Time Hand Gesture Recognition Based on Deep Learning YOLOv3 Model. *Applied Sciences* 11, 9 (2021). <https://doi.org/10.3390/app11094164>
- Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. 2015. ImageNet Large Scale Visual Recognition Challenge. arXiv:1409.0575 [cs.CV]
- Leslie N. Smith. 2017. Cyclical Learning Rates for Training Neural Networks. arXiv:1506.01186 [cs.CV]
- Fan Zhang, Valentin Bazarevsky, Andrey Vakunov, Andrei Tkachenka, George Sung, Chuo-Ling Chang, and Matthias Grundmann. 2020. MediaPipe Hands: On-device Real-time Hand Tracking. arXiv:2006.10214 [cs.CV]